
SOUTENANCE

GESTION DE STOCK

DAHMOUN Sarah

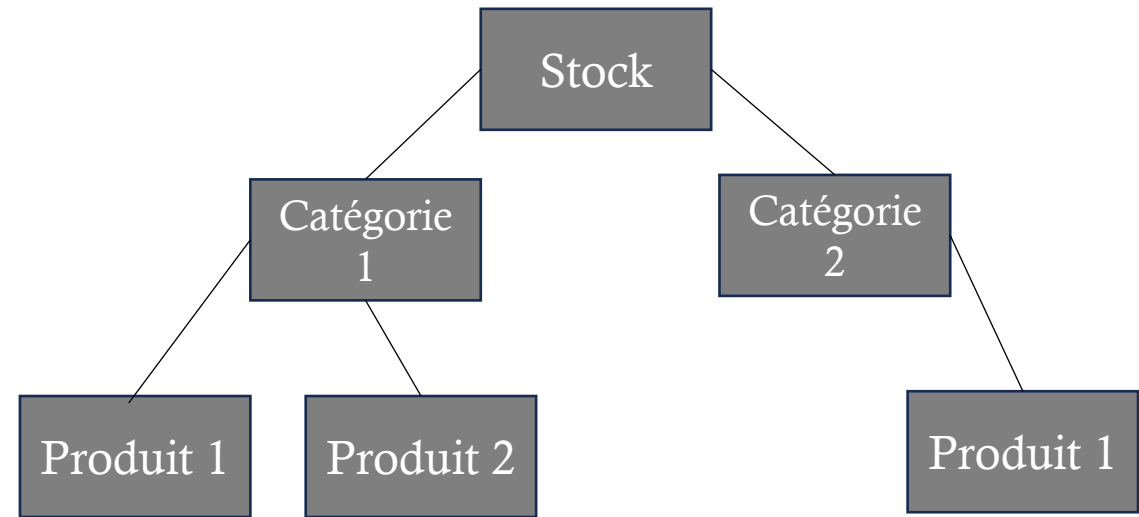
SINACER Tedj El Moulk



SOMMAIRE

1. Le sujet
2. La stratégie adoptée
 1. Vision globale
 2. Choix de la structure de données
 3. Types structurés
3. Les fonctions
 1. Algorithmes de tri
 2. Actions sur les produits
 3. Actions sur les catégories
 4. Gestion des fichiers

LE SUJET: GESTION DE STOCK



Fruits	Banane	200	1
Fruits	Orange	900	2
Legumes	Poivron	200	1
Legumes	Oignon	240,89999	28
Boulangerie	Pain	1,56	34
Boulangerie	Croissant	1.87	65



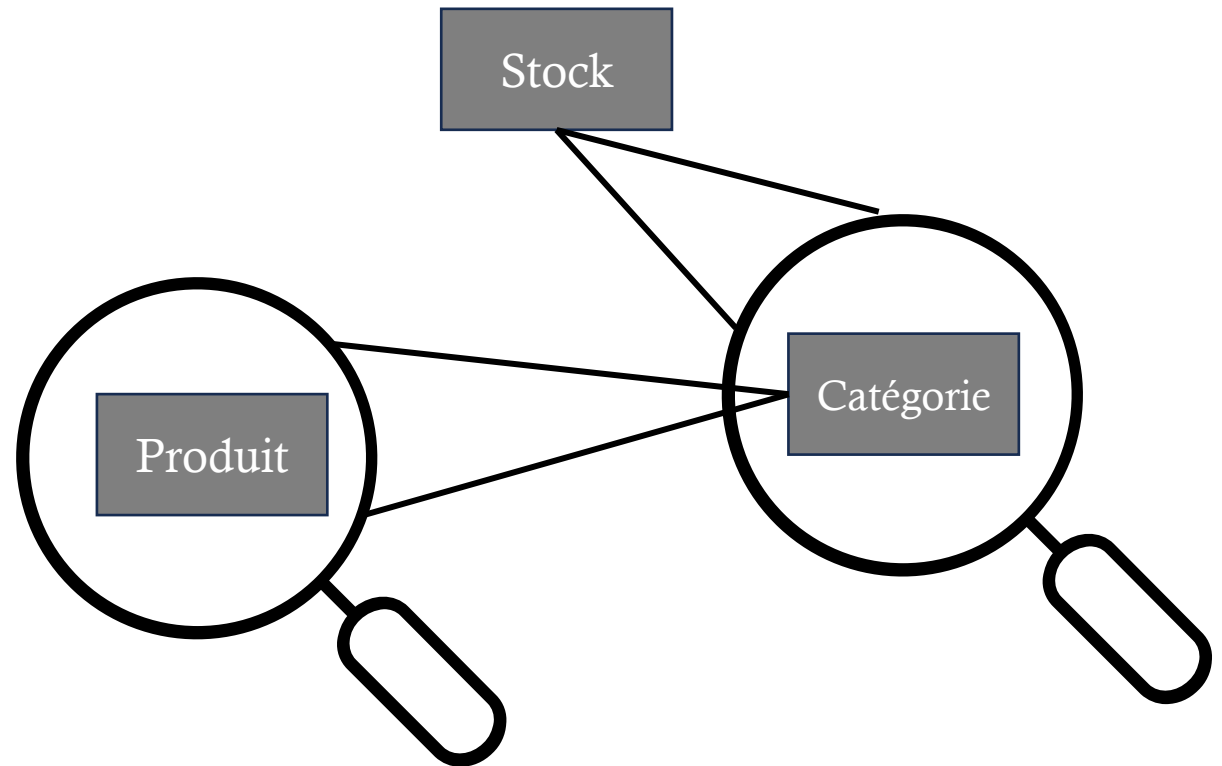
LE SUJET: GESTION DE STOCK

Fonctionnalités :

- Ajout de catégories
- Suppression de catégories
- Modification de catégories
 - Tris de produits
 - Ajout/Suppression de produits
 - Mise à jour des prix/quantités des produits

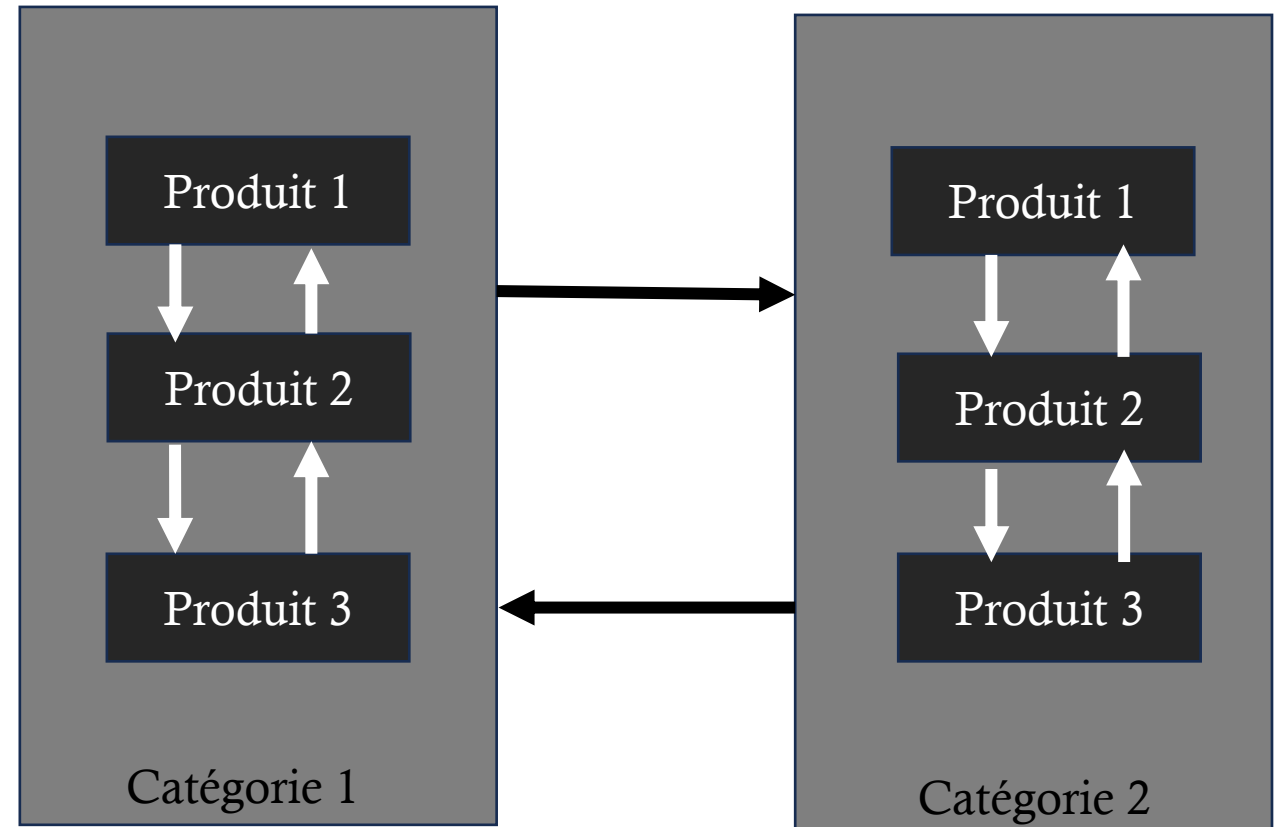
LA STRATÉGIE ADOPTÉE

La vision globale



LA STRATÉGIE ADOPTÉE

Choix de la structure de données : la liste chaînée



- ✓ Très extensible
- ✓ Espace mémoire dispersé

LA STRATÉGIE ADOPTÉE

Types structurés

```
typedef struct {  
    char nom_produit[SIZE];  
    float prix_produit;  
    int quantite_produit;  
} PRODUIT;
```

- Une structure qui représente un produit

```
typedef struct {  
    PRODUIT produit;  
    struct _NODE_P* next;  
    struct _NODE_P* previous;  
} NODE_P;
```

- Une structure qui représente un nœud dans la liste chaînée des produits

```
typedef struct {  
    NODE_P* first;  
    NODE_P* last;  
} LISTE_P;
```

- Une structure qui représente la liste chaînée produits (la catégorie) dans son ensemble

LA STRATÉGIE ADOPTÉE

Types structurés

```
typedef struct
{
    char nom_categorie[SIZE];
    LISTE_P* list_produits;
}CATEGORIE;
```

- Une structure qui représente une catégorie

```
typedef struct
{
    CATEGORIE categorie;
    struct _NODE_C* next;
    struct _NODE_C* previous;
}NODE_C;
```

- Une structure qui représente un nœud dans la liste chaînée des catégories

```
typedef struct
{
    NODE_C* first;
    NODE_C* last;
}LISTE_C;
```

- Une structure qui représente la liste chaînée des catégories (Le stock) dans son ensemble

LES FONCTIONS

Algorithmes de Tri



```
void tri_alphabetique(CATEGORIE* categorie)
{
    NODE_P* current = categorie->list_produits->first;
    NODE_P* min = categorie->list_produits->first;
    NODE_P* node_next = current->next;
    PRODUIT temp = { "", 0.0, 0 };

    while (current != NULL)
    {
        min = current;
        node_next = current->next;

        while (node_next != NULL)
        {
            if (strcoll(min->produit.nom_produit, node_next->produit.nom_produit) > 0)
            {
                min = node_next;
            }
            node_next = node_next->next;
        }

        if (min != current)
        {
            temp = current->produit;
            current->produit = min->produit;
            min->produit = temp;
        }
        current = current->next;
    }
    print_produits(*categorie);
}
```

LES FONCTIONS

Actions sur les produits

```
void suppression_produit(CATEGORIE* categorie)
{
    char produit[SIZE];

    NODE_P* node = categorie->list_produits->first;
    NODE_P* supp;
    NODE_P* aux;
    NODE_P* aux1;

    printf("Entrer le nom du produit : \n");
    scanf("%s", produit);

    while (node != NULL)
    {
        if (strcmp(node->produit.nom_produit, produit) == 0)
        {
            supp = node;

            if (node == categorie->list_produits->first)
            {
                node = node->next;
                categorie->list_produits->first = node;
                if (node != NULL)
                {
                    node->previous = NULL;
                }
            }
        }
    }
}
```

```
    else if (node == categorie->list_produits->last)
    {
        node = node->previous;
        categorie->list_produits->last = node;

        if (node != NULL)
        {
            node->next = NULL;
        }
    }
    else
    {
        aux = node->previous;
        aux->next = node->next;
        aux1 = node->next;
        aux1->previous = node->previous;
    }
    node = node->next;
}

print_produits(*categorie);
```

LES FONCTIONS

Actions sur les catégories

```
void ajout_categorie(LISTE_C* liste_categorie)
{
    char nvl_categorie[SIZE];
    int i = 2;

    NODE_C* node = (NODE_C*)malloc(sizeof(NODE_C));

    NODE_C* dernier = liste_categorie->last;

    //creation de la nvl categorie
    node->next = NULL;
    CATEGORIE nouvelle_categorie;
    printf("Entrer le nom de la nouvelle categorie : \n");
    scanf("%s",nvl_categorie);

    strcpy(nouvelle_categorie.nom_categorie,nvl_categorie);

    nouvelle_categorie.list_produits = (LISTE_P*)malloc(sizeof(LISTE_P));
    nouvelle_categorie.list_produits->first = NULL;
    nouvelle_categorie.list_produits->last = NULL;

    /* on ajoute le node a la fin de notre liste de categorie */
    if (node == NULL)
    {
        printf("ERREUR \n : ");
        return;
    }
}
```

```
node->categorie = nouvelle_categorie;
node->next = NULL;

if (dernier != NULL)
{
    dernier->next = node;
    node->previous = dernier;
}
else
{
    /* Si la liste est vide, le nouveau
    noeud devient le premier noeud */
    liste_categorie->first = node;
    node->previous = NULL;
}

liste_categorie->last = node;

while (i != 0) /* Boucle permettant de remplir
                la catégorie avec des produits */
{
    ajout_produit(&nouvelle_categorie);
    printf("Voulez-vous rajouter un autre produit a cette categorie?\n");
    printf("Tapez '1' pour 'oui'\n");
    printf("Tapez '0' pour 'non'\n");
    scanf("%d",&i);
}
```

LES FONCTIONS

Gestions de fichiers

1. Fonction qui permet de charger les informations stockées dans un fichier CSV dans une structure de données de type LISTE_C

- Création dynamique de nouveaux nœuds de catégories et produits
- Utilisation de « fgets » pour lire chaque ligne du fichier
- Utilisation de « strtok » pour diviser chaque ligne en tokens séparés par des points-virgules
- le premier token est le nom de la catégorie. Si ce nom est différent du précédent une nouvelle catégorie est créée

```
while (fgets(ligne, SIZE, fichier) != NULL)
/*Récupère la ligne et permet de passer à la ligne suivante */
{
    NODE_C* node_c = (NODE_C*)malloc(sizeof(NODE_C));
    token = strtok(ligne, ";");
/*Récupère les éléments d'une ligne séparés par des ; */
    if (token != NULL)
    {
        if (strcmp(token, categorie_precedente) != 0)
/* Comparaison du première éléments d'une ligne avec la categorie précédente */
        {
            //Nouvelle catégorie détectée
            strcpy(categorie_precedente, token);

            // Créer une nouvelle catégorie et créer une liste de produit
        }
    }
}
```

LES FONCTIONS

Gestions de fichiers

2. Fonction qui enregistre les données du stock dans un fichier CSV.

- La fonction ouvre un fichier en mode écriture
- Elle parcourt chaque nœud de catégorie dans la liste chaînée de catégorie
- Pour chaque catégorie elle parcourt chaque nœud de produit dans la liste chaînée de produit
- Pour chaque produit elle écrit les informations utilisant « fprintf » en respectant le format spécifié
- Le fichier est fermé avec « fclose »

```
void saveStock(LISTE_C* liste_categorie, const char* nom_fichier)
{
    FILE* fichier = fopen(nom_fichier, "w");

    NODE_C* node_c = (NODE_C*)malloc(sizeof(NODE_C));
    NODE_P* node_p = (NODE_P*)malloc(sizeof(NODE_P));
    node_c = liste_categorie->first;

    while (node_c != NULL)
    {
        node_p = node_c->categorie.list_produits->first;
        while (node_p != NULL)
        {
            fprintf(fichier, "%s;%s;%f;%d;\n", node_c->categorie.nom_categorie,
node_p->produit.nom_produit, node_p->produit.prix_produit, node_p->produit.quantite_produit);

            node_p = node_p->next;
        }
        node_c = node_c->next;
    }
    fclose(fichier);
}
```